

OPTION B INSTITUTE FAMILY

M03 - ADVANCED SQL + POSTGRESQL / DATABASE THINKING

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

16 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE03-01 - INNER JOIN and LEFT JOIN	5
DE03-02 - Cardinality, row multiplication and join validation	7
DE03-03 - UNION and UNION ALL	9
DE03-04 - String functions for cleaning and standardization	11
DE03-05 - Dates, timestamps and time analysis	13
DE03-06 - Subqueries and nested logic	15
DE03-07 - CTEs and readable multi-step SQL	17
DE03-08 - Window functions and PARTITION BY	19
DE03-09 - ROW_NUMBER, ranking and LAG/LEAD	21
DE03-10 - Advanced aggregation and multi-grain reasoning	23
DE03-11 - PostgreSQL setup, schemas and dialect awareness	25
DE03-12 - Constraints and transactions	27
DE03-13 - Indexes and EXPLAIN basics	29
DE03-14 - Views and reusable database logic	31
DE03-15 - Integrated unseen SQL challenge	33
DE03-16 - Timed SQL interview set and explanation	35
Module mastery checklist + revision quick reference	37

Prerequisite / tool map

Control joins, multi-step SQL, windows, PostgreSQL concepts and query reasoning.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE03-01 - INNER JOIN and LEFT JOIN

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Join related tables while choosing INNER vs LEFT from the business rule and stating the result grain before trusting the output.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

A JOIN combines rows through a relationship. INNER JOIN keeps matches only.

5. New words before we use them

A JOIN combines rows through a relationship. INNER JOIN keeps matches only. LEFT JOIN preserves every left-side row and supplies NULLs when the right side has no match.

6. Useful video / source

Source: Alex The Analyst — SQL joins

Assigned scope: 51:10–1:08:04

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT o.order_id, o.customer_id, c.customer_name, c.tier FROM sales_orders o JOIN customers c ON c.customer_id=o.customer_id; SELECT o.order_id, r.return_id FROM sales_orders o LEFT JOIN returns r ON r.order_id=o.order_id; Run both joins. Before each one, say the left input grain, right input grain and intended output grain. Count unmatched return rows in the LEFT JOIN.

8. Expected result / what you should see

- Customer join returns 30 order rows because customers is unique by customer_id. LEFT JOIN returns all 30 orders; exactly 5
- orders have a return_id in this lab.
- Recover when it goes wrong
- If rows disappear unexpectedly, check whether INNER JOIN was used where the business rule needed unmatched rows
- preserved. If rows multiply, inspect right-side key uniqueness before adding DISTINCT.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

List Completed West orders with order_id, customer_name, tier and net_sales. Then LEFT JOIN returns and show all Completed orders with optional return_id.

13. Validate

- No non-West/non-Completed rows in the first query. Second query preserves the Completed order count; returned
- orders have return_id and others are NULL.
- Customer join returns 30 order rows because customers is unique by customer_id. LEFT JOIN returns all 30 orders; exactly 5
- orders have a return_id in this lab.
- Recover when it goes wrong
- If rows disappear unexpectedly, check whether INNER JOIN was used where the business rule needed unmatched rows
- preserved. If rows multiply, inspect right-side key uniqueness before adding DISTINCT.

14. Teach it back

Explain INNER vs LEFT using one business example and state what unmatched rows mean.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-02

DE03-02 - Cardinality, row multiplication and join validation

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Predict one-to-one/one-to-many join behavior and detect row multiplication before it corrupts counts or money.

3. Prerequisite / starting state

Prerequisite: DE03-01 mastered with saved evidence.

4. Picture it in your head

Cardinality describes how many rows on one side can match the other. A syntactically valid join can still be logically wrong.

5. New words before we use them

Cardinality describes how many rows on one side can match the other. A syntactically valid join can still be logically wrong. Validate row count, key uniqueness and at least one numeric total before and after risky joins.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 4:16:18 — Data Modeling & JOINS

How to use it: Use the join/modeling block for the visual mental model. Course task remains authoritative for cardinality, row multiplication and validation.

Note: Support source because cardinality validation depth is course-owned.

[Open source](#)

7. Follow with me once

`SELECT COUNT(*) FROM sales_orders; -- 30`
`SELECT COUNT(*) FROM sales_orders o LEFT JOIN customer_contacts cc ON cc.customer_id=o.customer_id; -- 35`
 Join sales_orders to customer_contacts by customer_id. Find order_ids that appear more than once and connect the extra rows to customers having multiple contact types.

8. Expected result / what you should see

- Order rows = 30 before; 35 after the raw contact join. The five extra rows are expected fan-out from multi-contact customers,
- not five new orders.
- Recover when it goes wrong
- Do not repair fan-out with DISTINCT until you know the desired output grain. Pre-aggregate or choose one right-side row
- when the business question requires one row per order.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Build a one-row-per-customer `contact_count` CTE/subquery, join it to orders, and prove the order row count stays 30.

13. Validate

- Output key `order_id` remains unique; row count is 30; `contact_count` can exceed 1 without duplicating orders.
- Order rows = 30 before; 35 after the raw contact join. The five extra rows are expected fan-out from multi-contact customers,
- not five new orders.
- Recover when it goes wrong
- Do not repair fan-out with `DISTINCT` until you know the desired output grain. Pre-aggregate or choose one right-side row
- when the business question requires one row per order.

14. Teach it back

Explain why a valid foreign-key join can still inflate metrics.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-03

DE03-03 - UNION and UNION ALL

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Stack compatible result sets vertically and choose UNION vs UNION ALL intentionally.

3. Prerequisite / starting state

Prerequisite: DE03-02 mastered with saved evidence.

4. Picture it in your head

UNION ALL appends compatible result rows and keeps duplicates. UNION additionally removes duplicate result rows.

5. New words before we use them

UNION ALL appends compatible result rows and keeps duplicates. UNION additionally removes duplicate result rows. UNION adds rows; JOIN adds columns.

6. Useful video / source

Source: Alex The Analyst — UNION

Assigned scope: 1:08:04–1:15:10

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id AS event_id, customer_id, order_date AS event_date, 'Order' AS event_type FROM sales_orders WHERE status='Completed' UNION ALL SELECT ticket_id, customer_id, DATE(created_at), 'Ticket' FROM support_tickets; Build the combined order/ticket activity stream. Add ORDER BY event_date,event_id after the UNION ALL.

8. Expected result / what you should see

- The stream has 27 Completed-order events plus 10 ticket events = 37 rows because the event_id/type design keeps them
- distinct.
- Recover when it goes wrong
- If UNION fails, compare branch column counts and types. If a duplicate unexpectedly disappears, check whether UNION was
- used where UNION ALL was intended.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Create a second two-branch result with intentionally duplicated labels, compare UNION with UNION ALL, and record the row-count difference.

13. Validate

- You can explain exactly which duplicate combination UNION removed and why.
- The stream has 27 Completed-order events plus 10 ticket events = 37 rows because the event_id/type design keeps them
- distinct.
- Recover when it goes wrong
- If UNION fails, compare branch column counts and types. If a duplicate unexpectedly disappears, check whether UNION was
- used where UNION ALL was intended.

14. Teach it back

Explain UNION vs UNION ALL vs JOIN in one minute.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-04

DE03-04 - String functions for cleaning and standardization

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use TRIM/UPPER/LOWER/CONCAT and string matching without hiding the original value or overwriting source data.

3. Prerequisite / starting state

Prerequisite: DE03-03 mastered with saved evidence.

4. Picture it in your head

String functions can create cleaned/display values in a query. Keep the raw field beside transformed output while validating.

5. New words before we use them

String functions can create cleaned/display values in a query. Keep the raw field beside transformed output while validating. A presentation cleanup is not proof that source data is correct.

6. Useful video / source

Source: Alex The Analyst — string functions

Assigned scope: 1:15:10–1:26:39

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT customer_id, customer_name, UPPER(customer_name) AS name_upper,
CONCAT(customer_name, ' - ', tier) AS label FROM customers WHERE customer_name LIKE '%Retail%';
Run the example and identify every Retail customer. Add a TRIM demo on a literal string with spaces before touching source fields.

8. Expected result / what you should see

- Expected names include Ravi Retail, Om Retail, Ishita Retail, Saanvi Retail and Sara Retail.
- Recover when it goes wrong
- If a cleanup rule changes meaning, keep it as a derived value and investigate rather than updating the source. Do not use
- string cleanup as a substitute for a data contract.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Create `customer_name_clean` and a display label while preserving raw `customer_name`. Filter names containing Stores or Traders.

13. Validate

- Raw and transformed names remain visible; the filter returns only names matching the intended text rule.
- Expected names include Ravi Retail, Om Retail, Ishita Retail, Saanvi Retail and Sara Retail.
- Recover when it goes wrong
- If a cleanup rule changes meaning, keep it as a derived value and investigate rather than updating the source. Do not use
- string cleanup as a substitute for a data contract.

14. Teach it back

Explain safe query-time standardization versus destructive source cleanup.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-05

DE03-05 - Dates, timestamps and time analysis

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Calculate date differences, extract time parts and explain why date, datetime/timestamp and timezone meaning must be deliberate.

3. Prerequisite / starting state

Prerequisite: DE03-04 mastered with saved evidence.

4. Picture it in your head

Dates represent calendar dates; timestamps/datetimes represent moments or local date-times depending on the system. DATEDIFF can answer elapsed-day questions in this MySQL lab.

5. New words before we use them

Dates represent calendar dates; timestamps/datetimes represent moments or local date-times depending on the system. DATEDIFF can answer elapsed-day questions in this MySQL lab. Real systems need an explicit timezone contract before comparing event times.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 10:16:15 — Functions: Date, Text & NULL

How to use it: Watch the date/time portion, reproduce examples, then solve the course timestamp/time analysis independently.

Note: Current 2026 DE-focused SQL teaching.

[Open source](#)

7. Follow with me once

SELECT o.order_id, o.order_date, r.return_date, DATEDIFF(r.return_date,o.order_date) AS days_to_return FROM sales_orders o JOIN returns r ON r.order_id=o.order_id ORDER BY days_to_return DESC; Calculate days_to_return for all returned orders and inspect YEAR/MONTH/DAY on order_date.

8. Expected result / what you should see

- Five returned orders are present; every days_to_return is non-negative in the supplied lab.
- Recover when it goes wrong
- If a calculation mixes DATE and DATETIME unexpectedly, inspect types. Never assume a timezone for a timezone-less
- value unless the source contract says so.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Summarize Completed net_sales by order_date and identify the dates with the highest daily total. Explain what would change if order timestamps arrived from multiple timezones.

13. Validate

- Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed.
- Five returned orders are present; every days_to_return is non-negative in the supplied lab.
- Recover when it goes wrong
- If a calculation mixes DATE and DATETIME unexpectedly, inspect types. Never assume a timezone for a timezone-less
- value unless the source contract says so.

14. Teach it back

Explain date vs timestamp and one timezone failure mode.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-06

DE03-06 - Subqueries and nested logic

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use scalar/list subqueries while keeping each nested level independently testable.

3. Prerequisite / starting state

Prerequisite: DE03-05 mastered with saved evidence.

4. Picture it in your head

A subquery is a query used inside another query. Debug the inner level first.

5. New words before we use them

A subquery is a query used inside another query. Debug the inner level first. Deep nesting is not a badge of sophistication; readable grain and validation matter more.

6. Useful video / source

Source: Alex The Analyst — subqueries

Assigned scope: 1:35:16–1:45:58

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT order_id, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders WHERE status='Completed' AND qty*unit_price*(1-discount_pct) > (SELECT AVG(qty*unit_price*(1-discount_pct)) FROM sales_orders WHERE status='Completed'); Run the inner average independently, then the full above-average Completed-order query.

8. Expected result / what you should see

- The inner subquery returns one scalar. Outer rows are all Completed and each net_sales is above that scalar.
- Recover when it goes wrong
- If the subquery returns multiple rows where one value is required, restate what the inner query should represent. Use
- IN/EXISTS when a set/existence question is intended.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Find customers whose Completed net_sales total is above the average customer Completed total. Build/validate the customer-total layer first.

13. Validate

- State inner and outer grain separately and reconcile customer totals to overall Completed net_sales.
- The inner subquery returns one scalar. Outer rows are all Completed and each net_sales is above that scalar.
- Recover when it goes wrong
- If the subquery returns multiple rows where one value is required, restate what the inner query should represent. Use
- IN/EXISTS when a set/existence question is intended.

14. Teach it back

Explain why you test a subquery independently before embedding it.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-07

DE03-07 - CTEs and readable multi-step SQL

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Break multi-step SQL into named CTEs and state the grain at each stage.

3. Prerequisite / starting state

Prerequisite: DE03-06 mastered with saved evidence.

4. Picture it in your head

A CTE gives a query step a name. Its primary value here is readability and inspectable grain, not automatic performance.

5. New words before we use them

A CTE gives a query step a name. Its primary value here is readability and inspectable grain, not automatic performance. Each CTE should have a clear purpose and row meaning.

6. Useful video / source

Source: Alex The Analyst — CTEs

Assigned scope: 1:58:55–2:09:07

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

WITH order_sales AS (SELECT order_id,customer_id,region,status, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders), customer_sales AS (SELECT customer_id,SUM(net_sales) AS customer_net FROM order_sales WHERE status='Completed' GROUP BY customer_id) SELECT * FROM customer_sales ORDER BY customer_net DESC; Build order_sales then customer_sales. Run each stage separately during development and record row counts.

8. Expected result / what you should see

- order_sales = one row/order, 30 rows. customer_sales = one row/customer with Completed activity. Its summed
- customer_net reconciles to Completed net sales.
- Recover when it goes wrong
- If a CTE changes grain unexpectedly, stop and query only that CTE. Do not keep stacking later logic on an unvalidated
- intermediate result.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Rewrite your DE03-06 customer-above-average solution with named CTEs and add a final validation CTE/result.

13. Validate

- Every CTE has a written grain comment and totals reconcile.
- `order_sales` = one row/order, 30 rows. `customer_sales` = one row/customer with Completed activity. Its summed
- `customer_net` reconciles to Completed net sales.
- Recover when it goes wrong
- If a CTE changes grain unexpectedly, stop and query only that CTE. Do not keep stacking later logic on an unvalidated
- intermediate result.

14. Teach it back

Explain CTE value without claiming it is always faster.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-08

DE03-08 - Window functions and PARTITION BY

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Add group context to detail rows with window functions while distinguishing window grain from GROUP BY grain.

3. Prerequisite / starting state

Prerequisite: DE03-07 mastered with saved evidence.

4. Picture it in your head

GROUP BY collapses rows to the group grain. A window calculation can preserve detail rows and attach group-level context.

5. New words before we use them

GROUP BY collapses rows to the group grain. A window calculation can preserve detail rows and attach group-level context. PARTITION BY defines independent calculation groups.

6. Useful video / source

Source: Alex The Analyst — window functions

Assigned scope: 1:45:58–1:58:55

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

WITH x AS (SELECT order_id,region,qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders WHERE status='Completed') SELECT order_id,region,net_sales, AVG(net_sales) OVER(PARTITION BY region) AS region_avg_sales FROM x; Add region average net_sales beside every Completed order. Compare row count to a GROUP BY region query.

8. Expected result / what you should see

- Window result preserves 27 Completed detail rows; GROUP BY region produces 4 rows.
- Recover when it goes wrong
- If detail rows disappear, check whether GROUP BY was used instead of a window. If partition totals are wrong, inspect the
 - partition key and eligibility filter.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Add each customer's Completed order count and customer net-sales total beside each of their Completed orders.

13. Validate

- Detail row count stays 27; a separate GROUP BY customer query matches the window totals.
- Window result preserves 27 Completed detail rows; GROUP BY region produces 4 rows.
- Recover when it goes wrong
- If detail rows disappear, check whether GROUP BY was used instead of a window. If partition totals are wrong, inspect the
- partition key and eligibility filter.

14. Teach it back

Explain PARTITION BY and why windows preserve detail.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-09

DE03-09 - ROW_NUMBER, ranking and LAG/LEAD

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use deterministic ROW_NUMBER/ranking and LAG/LEAD for latest-row, top-N and previous/next-row analysis.

3. Prerequisite / starting state

Prerequisite: DE03-08 mastered with saved evidence.

4. Picture it in your head

ROW_NUMBER assigns a unique sequence based on the ORDER BY you define. RANK/DENSE_RANK treat ties differently.

5. New words before we use them

ROW_NUMBER assigns a unique sequence based on the ORDER BY you define. RANK/DENSE_RANK treat ties differently. LAG/LEAD reaches a previous/next row within a partition/order.

6. Useful video / source

Source: Alex The Analyst — ranking/window functions

Assigned scope: 1:45:58–1:58:55

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

WITH t AS (SELECT ticket_id, customer_id, created_at, status, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY created_at DESC, ticket_id DESC) AS rn FROM support_tickets) SELECT * FROM t WHERE rn=1; Return latest support ticket per customer. Then add LAG(order_date) to each customer's orders.

8. Expected result / what you should see

- Latest-ticket query returns one row per customer who has tickets. Sequence has an explicit ticket_id tie-breaker.
- Recover when it goes wrong
- If latest-row output is nondeterministic on equal timestamps, add a stable tie-breaker. If LAG crosses customers, add
- PARTITION BY customer_id.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Rank Completed orders by net_sales within region and return top 2 per region with deterministic order_id tie-breaker.

13. Validate

- At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately.
- Latest-ticket query returns one row per customer who has tickets. Sequence has an explicit ticket_id tie-breaker.
- Recover when it goes wrong
- If latest-row output is nondeterministic on equal timestamps, add a stable tie-breaker. If LAG crosses customers, add
- PARTITION BY customer_id.

14. Teach it back

Explain ROW_NUMBER vs RANK and one LAG use case.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-10

DE03-10 - Advanced aggregation and multi-grain reasoning

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Combine metrics from different source grains without multiplying them and defend the chosen final grain.

3. Prerequisite / starting state

Prerequisite: DE03-09 mastered with saved evidence.

4. Picture it in your head

Orders, returns and tickets are not automatically the same grain. Compute each metric at a stable grain first, then join summaries at the intended output grain.

5. New words before we use them

Orders, returns and tickets are not automatically the same grain. Compute each metric at a stable grain first, then join summaries at the intended output grain. COUNT(DISTINCT ...) can sometimes protect a count, but it does not magically repair multiplied monetary sums.

6. Useful video / source

Source: Luke Barousse — SQL for Data Analytics

Assigned scope: 37:44–1:04:47 — aggregation through statistical aggregation; 1:04:47 onward segmentation

How to use it: Use these blocks to strengthen aggregation fluency. The Teaching Book adds multi-grain engineering reasoning and validation.

Note: Video target/title confirmed; exact engineering multi-grain proof remains course-owned.

[Open source](#)

7. Follow with me once

WITH order_region AS (SELECT region,COUNT(*) AS orders_count,SUM(qty*unit_price*(1-discount_pct)) AS net_sales FROM sales_orders WHERE status='Completed' GROUP BY region), return_region AS (SELECT o.region,COUNT(*) AS returned_orders FROM returns r JOIN sales_orders o ON o.order_id=r.order_id GROUP BY o.region) SELECT o.*,COALESCE(r.returned_orders,0) AS returned_orders FROM order_region o LEFT JOIN return_region r USING(region); Build order and return metrics separately by region, then combine them. Explain why directly joining orders to tickets by customer_id could multiply both sides.

8. Expected result / what you should see

- Final result is exactly one row/region. Sum of regional net_sales reconciles to Completed net_sales. Returned-order counts
- sum to 5.
- Recover when it goes wrong
- If totals inflate after a join, inspect source/result grain and pre-aggregate the many-side metrics before joining. Do not hide the
- problem with DISTINCT on the final output.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Create region metrics: Completed orders, Completed net_sales, returned orders, ticket count. Use separate grain-safe CTEs and combine at one row/region.

13. Validate

- 4 final rows; each component total reconciles independently; document any customer-to-region assumption.
- Final result is exactly one row/region. Sum of regional net_sales reconciles to Completed net_sales. Returned-order counts
- sum to 5.
- Recover when it goes wrong
- If totals inflate after a join, inspect source/result grain and pre-aggregate the many-side metrics before joining. Do not hide the
- problem with DISTINCT on the final output.

14. Teach it back

Explain why multi-grain analysis is an engineering correctness problem, not just SQL style.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Video route: Luke Barousse - SQL for Data Analytics - Intermediate Course + Project Segment: 37:44-1:04:47 aggregation; 1:04:47-1:25:28 advanced segmentation <https://www.youtube.com/watch?v=QKIGsShyEsQ> Verified 2026-09-10; corrected timestamp/URL parity. Book owns multi-grain proof. Exact next route: DE03-11

DE03-11 - PostgreSQL setup, schemas and dialect awareness

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Install/verify PostgreSQL when this lesson begins, connect with psql or pgAdmin, create/use a schema, and recognize that SQL dialect/tooling differs from MySQL.

3. Prerequisite / starting state

Prerequisite: DE03-10 mastered with saved evidence.

4. Picture it in your head

PostgreSQL is a separate database system from MySQL. Common SQL concepts transfer, but setup, functions, types and commands can differ.

5. New words before we use them

PostgreSQL is a separate database system from MySQL. Common SQL concepts transfer, but setup, functions, types and commands can differ. A schema is a namespace inside a PostgreSQL database.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL setup

Assigned scope: 0:00–17:12

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

SELECT version(); CREATE SCHEMA IF NOT EXISTS m03pg; SET search_path TO m03pg, public; SELECT current_schema(); Install/verify the current supported PostgreSQL release from the official project when you reach this lesson. Create database m03_de (or use an existing course DB), run M03_POSTGRESQL_MINI_LAB_SETUP.sql, then run SELECT version() and SHOW search_path.

8. Expected result / what you should see

- You can connect successfully, identify the server version you actually observe, and query m03pg.orders. The mini-lab reports
- 8 customers and 12 orders.
- Recover when it goes wrong
- If psql/pgAdmin cannot connect, stop at connection setup rather than changing SQL. Verify server service,
- host/port/database/user and exact error. Do not install PostgreSQL earlier in M02.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

In a fresh query tab, set `search_path` and return `order_id`, `customer_id`, `status` from `m03pg.orders`. Write two syntax/function differences you have actually observed or verified between the MySQL and PostgreSQL labs.

13. Validate

- Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented.
- You can connect successfully, identify the server version you actually observe, and query `m03pg.orders`. The mini-lab reports
- 8 customers and 12 orders.
- Recover when it goes wrong
- If `psql/pgAdmin` cannot connect, stop at connection setup rather than changing SQL. Verify server service,
- `host/port/database/user` and exact error. Do not install PostgreSQL earlier in M02.

14. Teach it back

Explain database vs schema and why SQL portability is not perfect.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Video route: freeCodeCamp - PostgreSQL Tutorial for Beginners Segment: 0:00-17:12 | <https://www.youtube.com/watch?v=SpflwIAYaKk> Version guidance: PostgreSQL 18.6/current 18 verified 2026-09-10. PostgreSQL 19 Beta 3 is not the course baseline. Install only now, at DE03-11. Exact next route: DE03-12

DE03-12 - Constraints and transactions

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use primary/foreign/check/not-null constraints and prove COMMIT/ROLLBACK behavior without weakening integrity rules.

3. Prerequisite / starting state

Prerequisite: DE03-11 mastered with saved evidence.

4. Picture it in your head

Constraints make invalid states harder to store. A transaction groups changes so they can commit together or roll back.

5. New words before we use them

Constraints make invalid states harder to store. A transaction groups changes so they can commit together or roll back. When a constraint rejects a row, the correct response is to understand the rule/data - not remove the constraint to silence the error.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL constraints/transactions

Assigned scope: 2:29:24–3:25:04

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

BEGIN; UPDATE m03pg.orders SET status='Cancelled' WHERE order_id='P1001'; SELECT order_id,status FROM m03pg.orders WHERE order_id='P1001'; ROLLBACK; SELECT order_id,status FROM m03pg.orders WHERE order_id='P1001'; Run the rollback demonstration. Then deliberately attempt a duplicate order_id and an invalid status inside a safe transaction and capture the constraint errors.

8. Expected result / what you should see

- P1001 returns to its original status after ROLLBACK. Invalid/duplicate rows do not remain stored.
- Recover when it goes wrong
- If a failed statement aborts the PostgreSQL transaction, ROLLBACK before continuing. Do not drop constraints to make the
- test pass.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Insert one valid support_case inside BEGIN; inspect it; ROLLBACK; prove row count returns to the baseline. Explain which constraint protects each critical field.

13. Validate

- Baseline row count restored after rollback; no invalid row persists.
- P1001 returns to its original status after ROLLBACK. Invalid/duplicate rows do not remain stored.
- Recover when it goes wrong
- If a failed statement aborts the PostgreSQL transaction, ROLLBACK before continuing. Do not drop constraints to make the
- test pass.

14. Teach it back

Explain transaction, COMMIT, ROLLBACK and one integrity constraint in plain English.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Video A: freeCodeCamp / Amigoscode PostgreSQL - constraints and foreign keys 2:29:24-2:54:45 + 3:16:41-3:25:04 | <https://www.youtube.com/watch?v=qw--VYLpxG4> Video B: Harvard CS50 SQL Week 5 - transactions 1:02:29-1:15:57 | <https://cs50.harvard.edu/sql/weeks/5/> Verified 2026-09-10; book/lab controls PostgreSQL-specific mastery. Exact next route: DE03-13

DE03-13 - Indexes and EXPLAIN basics

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Create a query-driven index, read basic EXPLAIN evidence, and avoid claiming speedup that the small learning table cannot prove.

3. Prerequisite / starting state

Prerequisite: DE03-12 mastered with saved evidence.

4. Picture it in your head

An index is an additional structure that can help some lookups but costs storage/write maintenance. PostgreSQL chooses plans based on table size/statistics/cost.

5. New words before we use them

An index is an additional structure that can help some lookups but costs storage/write maintenance. PostgreSQL chooses plans based on table size/statistics/cost. On tiny course tables, the planner may still prefer a sequential scan.

6. Useful video / source

Source: Luke Barousse — SQL for Data Analytics

Assigned scope: 5:10:55-5:42:10 — EXPLAIN and query optimization

How to use it: Watch the EXPLAIN/optimization block, then use the course PostgreSQL task to compare plans and explain what changed.

Note: Video target confirmed; PostgreSQL-specific course lab remains the mastery proof.

[Open source](#)

7. Follow with me once

EXPLAIN SELECT * FROM m03pg.orders WHERE customer_id='PC003'; CREATE INDEX IF NOT EXISTS idx_m03_orders_customer_id ON m03pg.orders(customer_id); ANALYZE m03pg.orders; EXPLAIN SELECT * FROM m03pg.orders WHERE customer_id='PC003'; Capture EXPLAIN before and after the index. Compare plan nodes/cost/rows estimates you actually see; do not require an Index Scan on a 12-row table.

8. Expected result / what you should see

- Index exists in pg_indexes after creation. Before/after plans are saved. Any claim about planner choice matches observed
- output.
- Recover when it goes wrong
- If the plan still shows Seq Scan, that is not a failure on a tiny table. Explain table-size/cost reasoning; do not force planner
- settings just to manufacture an index scan.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Choose one repeated filter/join column from the mini-lab, justify whether an index is sensible, create it only if justified, and save before/after EXPLAIN plus one downside.

13. Validate

- Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.”
- Index exists in pg_indexes after creation. Before/after plans are saved. Any claim about planner choice matches observed
- output.
- Recover when it goes wrong
- If the plan still shows Seq Scan, that is not a failure on a tiny table. Explain table-size/cost reasoning; do not force planner
- settings just to manufacture an index scan.

14. Teach it back

Explain what EXPLAIN proves and what it does not prove.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Video route: Luke Barousse - SQL for Data Analytics - Intermediate Course + Project Segment: 5:10:55-5:42:10 EXPLAIN and query-optimization support <https://www.youtube.com/watch?v=QKIGsShyEsQ> Verified 2026-09-10; corrected timestamp/URL parity. PostgreSQL lab is mastery proof. Exact next route: DE03-14

DE03-14 - Views and reusable database logic

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Create a view for stable reusable query logic while explaining that a normal view stores a query definition, not an independent copy of the data.

3. Prerequisite / starting state

Prerequisite: DE03-13 mastered with saved evidence.

4. Picture it in your head

A normal PostgreSQL view gives a reusable named SELECT. It can simplify consumer queries and centralize logic, but it depends on underlying objects and permissions.

5. New words before we use them

A normal PostgreSQL view gives a reusable named SELECT. It can simplify consumer queries and centralize logic, but it depends on underlying objects and permissions. A view is not automatically a performance optimization or a materialized dataset.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use matching Views section; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
CREATE OR REPLACE VIEW m03pg.v_completed_order_value AS SELECT
o.order_id,o.order_date,o.customer_id,c.customer_name, SUM(i.qty*i.unit_price) AS gross_sales FROM
m03pg.orders o JOIN m03pg.customers c USING(customer_id) JOIN m03pg.order_items i
USING(order_id) WHERE o.status='Completed' GROUP BY
o.order_id,o.order_date,o.customer_id,c.customer_name; SELECT * FROM
m03pg.v_completed_order_value; Create the completed-order view, query it, then change one
underlying row inside a transaction and observe the view before rolling back.
```

8. Expected result / what you should see

- The view exposes only Completed orders and reflects underlying data because it is a normal view. Rollback returns results to
- baseline.
- Recover when it goes wrong
- If the view duplicates orders, inspect order_items aggregation and intended view grain. If consumers need a physical cached
- result, that is a different design topic; do not call a normal view materialized.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Create a view `m03pg.v_customer_order_summary` at one row/customer with `order_count` and `gross_sales`. State dependencies and one limitation.

13. Validate

- One row/customer with activity; sum of `gross_sales` matches an independent aggregate over eligible source rows.
- The view exposes only Completed orders and reflects underlying data because it is a normal view. Rollback returns results to
- baseline.
- Recover when it goes wrong
- If the view duplicates orders, inspect `order_items` aggregation and intended view grain. If consumers need a physical cached
- result, that is a different design topic; do not call a normal view materialized.

14. Teach it back

Explain normal view vs table and why views can improve maintainability without guaranteeing speed.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Video route: Luke Barousse - SQL for Data Analytics - Intermediate Course + Project Segment: 3:45:59-4:06:27 View Intro <https://www.youtube.com/watch?v=QKIGsShyEsQ> Verified 2026-09-10; old source had no Views chapter. Book/PostgreSQL lab authoritative. Exact next route: DE03-15

DE03-15 - Integrated unseen SQL challenge

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Combine grain/cardinality, CTE/window logic, validation and PostgreSQL database-thinking skills without tutorial support.

3. Prerequisite / starting state

Prerequisite: DE03-14 mastered with saved evidence.

4. Picture it in your head

This is an assessment lesson, not another tutorial. A genuine attempt is required before review.

5. New words before we use them

This is an assessment lesson, not another tutorial. A genuine attempt is required before review. You may use the database clients and the supplied source data, but keep videos, solution/review material and AI solving closed.

6. Useful video / source

Source: Independent practice / protected assessment

How to use it: Keep videos, answers and AI solving closed during the attempt. Use review only after saving a genuine attempt.

Note: No tutorial during this task by design. Attempt it independently, then use the locked review/remediation flow.

Disposition: COURSE_BUILT / BOOK_ONLY for this lesson; no filler link is required.

7. Follow with me once

No worked solution is shown before attempt. Read the challenge brief, state grains/eligibility, build in small validated steps and preserve failed attempts. No guided solution. Planning scaffold only: 1) state desired output grain; 2) list source grains; 3) identify fan-out risks; 4) build/validate stages; 5) save result + controls; 6) explain limitations.

8. Expected result / what you should see

- Challenge rubric requires correct grain, no unexplained multiplication, at least one CTE/window pattern, two independent
- validations, and one PostgreSQL object/transaction/plan evidence item.
- Recover when it goes wrong
- After a saved attempt, open only the matching protected review for weak competencies. Then close review and use the fresh
- retest variant.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Complete the M03 integrated challenge in 90 minutes. MySQL portion: build a one-row/customer priority table from orders + tickets without fan-out, with latest ticket and customer sales rank. PostgreSQL portion: prove one rollback, one justified index/EXPLAIN comparison and one reusable view.

13. Validate

- All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and
- PostgreSQL evidence is observed/saved.
- Challenge rubric requires correct grain, no unexplained multiplication, at least one CTE/window pattern, two independent
- validations, and one PostgreSQL object/transaction/plan evidence item.
- Recover when it goes wrong
- After a saved attempt, open only the matching protected review for weak competencies. Then close review and use the fresh
- retest variant.

14. Teach it back

Give a 3-minute defense: grain, biggest fan-out risk, validation, Postgres object choice and one limitation.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: DE03-16

DE03-16 - Timed SQL interview set and explanation

1. Why it matters

This matters because later Advanced SQL + PostgreSQL / Database Thinking work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Solve a fresh timed SQL set and explain choices clearly under interview-like constraints.

3. Prerequisite / starting state

Prerequisite: DE03-15 mastered with saved evidence.

4. Picture it in your head

Timed work tests retrieval and reasoning, not memorization. Correctness includes grain and validation.

5. New words before we use them

Timed work tests retrieval and reasoning, not memorization. Correctness includes grain and validation. A query that runs but answers the wrong question is not a pass.

6. Useful video / source

Source: Independent practice / protected assessment

How to use it: Keep videos, answers and AI solving closed during the attempt. Use review only after saving a genuine attempt.

Note: No tutorial during this task by design. Attempt it independently, then use the locked review/remediation flow.

Disposition: COURSE_BUILT / BOOK_ONLY for this lesson; no filler link is required.

7. Follow with me once

No answer is exposed before attempt. Use a 45-minute timer for the SQL set plus 10-15 minutes for explanation/validation notes. No guided solution. Time allocation suggestion: 5 min read/grain, 30 min queries, 10 min validation, then explanation.

8. Expected result / what you should see

- Pass target: $\geq 85/100$, no critical grain/cardinality/unsafe-transaction failure, and clear explanation of at least one
- window/CTE and one PostgreSQL database object.
- Recover when it goes wrong
- If one domain fails, repair only that domain and take a different timed set. Do not repeat the same questions as mastery
- evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Timed set: (1) diagnose a fan-out join; (2) top-2 Completed non-returned orders/region; (3) latest ticket/customer; (4) region net-sales + return rate safely; (5) explain transaction rollback and EXPLAIN evidence; (6) defend view vs table.

13. Validate

- Save SQL, result controls, time used, confidence, errors and explanation. Review remains closed until the attempt is
 - preserved.
- Pass target: $\geq 85/100$, no critical grain/cardinality/unsafe-transaction failure, and clear explanation of at least one
 - window/CTE and one PostgreSQL database object.
- Recover when it goes wrong
- If one domain fails, repair only that domain and take a different timed set. Do not repeat the same questions as mastery
- evidence.

14. Teach it back

Answer two random questions from the set aloud without reading your SQL.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Save runnable SQL or database evidence, the relevant row/grain/count/reconciliation check, one error or uncertainty, confidence 0-5, and your own-words explanation. If review materially helped repair the attempt, close review and solve a changed retry before Mastered. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.